

SALESTORM

High-Scale E-Commerce Flash Sale System

SYSTEM DESIGN & AI-ASSISTED PROTOTYPE DOCUMENTATION

SYSCRAFTERS 2026 • Design-First, AI-Assisted System Design Hackathon

Prepared as an implementation-ready design blueprint and prototype specification.

Document status: Design baseline • Prototype results to be filled after validation

Document Control

Item	Value
Project	SALESTORM – High-Scale E-Commerce Flash Sale Platform
Challenge	10,000 concurrent purchase requests competing for 100 units
Primary goal	Prevent overselling while maintaining reliable payment and order processing
Document purpose	Provide the design specification that guides prototype implementation and validation
Prototype role	Optional AI-assisted validation evidence; not a replacement for system design
Primary source	SALESTORM SYSCRAFTERS 2026 Design-First, AI-Assisted System Design Hackathon brief
Prepared by	[Team Name / Team Members]
Date	05 October 2026

Table of Contents

Use Word's automatic Table of Contents after opening the document: References → Table of Contents → Automatic Table.

1. Executive Summary

SALESTORM is a high-volume e-commerce flash-sale platform designed for a scenario in which a large number of customers compete for a small amount of inventory. The core challenge is to handle 10,000 simultaneous purchase requests for a product with only 100 available units without overselling, duplicate reservations or duplicate payments.

The proposed design uses controlled request admission, horizontally scalable services, a concurrency-safe inventory reservation mechanism, temporary reservations with expiry, idempotent payment processing, asynchronous recovery for downstream failures, and an explicit order lifecycle. The architecture separates business responsibilities into Product, Cart, Inventory/Reservation, Checkout, Payment, Order, Shipment and Notification capabilities, supported by caching, messaging, database, monitoring, logging and tracing.

The accompanying prototype is intentionally small. Its purpose is to validate the most critical design assumptions: concurrency safety, inventory consistency, idempotency, reservation expiry, payment failure handling and recovery when the Order Service is temporarily unavailable.

The hackathon brief states that design correctness and engineering judgement are more important than code volume. Therefore, the prototype is treated as evidence supporting the architecture rather than as the primary deliverable.

2. Problem Statement

During a flash sale, traffic can increase sharply within a short period. Product X has 100 available units while 10,000 customers may simultaneously select Buy Now. If inventory is read and updated without concurrency control, multiple requests can observe the same quantity and commit more sales than the actual stock.

The system must therefore provide a deterministic mechanism for controlling inventory contention, temporary reservation, safe payment processing, correct order state transitions and recovery from service or gateway failures.

3. Business Objectives

- Support sudden increases in customer traffic without system collapse.
- Prevent inventory overselling under concurrent purchase requests.
- Provide temporary inventory reservation with expiry and release.
- Ensure safe and idempotent payment processing.
- Maintain a consistent and traceable order lifecycle.
- Recover gracefully from service, database and payment failures.
- Produce a design that an engineering team can understand, implement and operate.

4. Scope

4.1 In Scope

- Flash-sale purchase flow from request arrival through inventory reservation, checkout, payment and order creation.
- Concurrency-safe reservation of limited inventory.
- Reservation expiry and release.
- Idempotent purchase and payment requests.
- Payment success, failure and timeout simulation.

- Order Service temporary unavailability and recovery.
- Database, API, LLD, reliability, security and observability design.
- AI-assisted prototype and validation simulation.

4.2 Out of Scope

- Full production implementation of every e-commerce capability.
- Real payment gateway integration.
- Real shipment carrier integration.
- Real customer notification provider integration.
- Proof of production-scale 500,000 requests/second capacity; that remains an architectural scaling target requiring production infrastructure and load testing.

5. Assumptions and Constraints

ID	Assumption / Constraint
A1	Product X begins with 100 available units.
A2	10,000 customers can issue purchase requests concurrently.
A3	A purchase reserves inventory before payment is finalized.
A4	Reservations have an expiry time and can be released.
A5	Payment requests can be repeated and therefore require idempotency.
A6	Order creation can temporarily fail after payment succeeds.
A7	The prototype uses a simulated payment gateway.
A8	The prototype validates correctness rather than production infrastructure capacity.
A9	AI-generated implementation must be reviewed and validated by the team.

6. Functional Requirements

ID	Requirement	Priority
FR-01	Customer can request purchase of a product.	High
FR-02	System checks and reserves available inventory atomically.	Critical
FR-03	System prevents inventory from becoming negative.	Critical
FR-04	System creates temporary reservations with expiry.	Critical
FR-05	Expired or failed reservations release inventory.	Critical
FR-06	System processes payment using an idempotent transaction reference.	Critical
FR-07	Duplicate purchase requests do not create duplicate reservations.	Critical
FR-08	Duplicate payment requests do not create duplicate transactions.	Critical
FR-09	Successful payment leads to order creation.	High

FR-10	Order creation is retried when the Order Service is temporarily unavailable.	High
FR-11	System reports OUT_OF_STOCK after all units are unavailable.	High
FR-12	System records critical business events for observability.	Medium

7. Non-Functional Requirements

Area	Requirement / Target
Concurrency	Handle 10,000 simultaneous purchase attempts through controlled contention.
Inventory	No more than 100 units may become successful sales for the 100-unit scenario.
Consistency	Inventory state must remain valid across reservation and sale transitions.
Scalability	Services should support horizontal scaling; architecture should reason about normal and flash-sale traffic.
Reliability	Retries, idempotency, circuit-breaker behaviour, dead-letter handling and reconciliation should be considered.
Security	Authentication/authorization, HTTPS, input validation, rate limiting, secure payment handling, audit logging and secrets management.
Observability	Metrics, structured logs, distributed tracing and alerts for critical failures.
Recovery	System must define recovery paths for database, payment and downstream service failures.

8. Success Criteria

Criterion	Expected Outcome
Concurrency	10,000 simultaneous requests can be handled through controlled contention and defined scaling behaviour.
Inventory	100 available units cannot result in more than 100 successful sales.
Reservation	Unpaid or expired reservations are released correctly.
Payment	Duplicate payment requests do not create duplicate transactions.
Order	Successful purchases eventually reach a valid order state.
Reliability	Defined recovery paths exist for service and gateway failures.
Scalability	Architecture supports horizontal scaling during peak demand.

9. System Context

The system context identifies customers, the SALESTORM platform and external dependencies. Customers initiate product discovery and purchase actions. The SALESTORM platform coordinates product, cart, inventory/reservation, checkout, payment and order workflows. External systems include a payment gateway and downstream fulfilment/notification providers.

Context diagram to create in Draw.io / Mermaid / PlantUML:

Customer → SALESTORM Platform → Payment Gateway
SALESTORM Platform → Shipment Provider
SALESTORM Platform → Notification Provider

Supporting platform infrastructure: CDN/WAF, Load Balancer, API Gateway, Cache, Message Broker, Database, Monitoring, Logging and Distributed Tracing.



10. High-Level Architecture

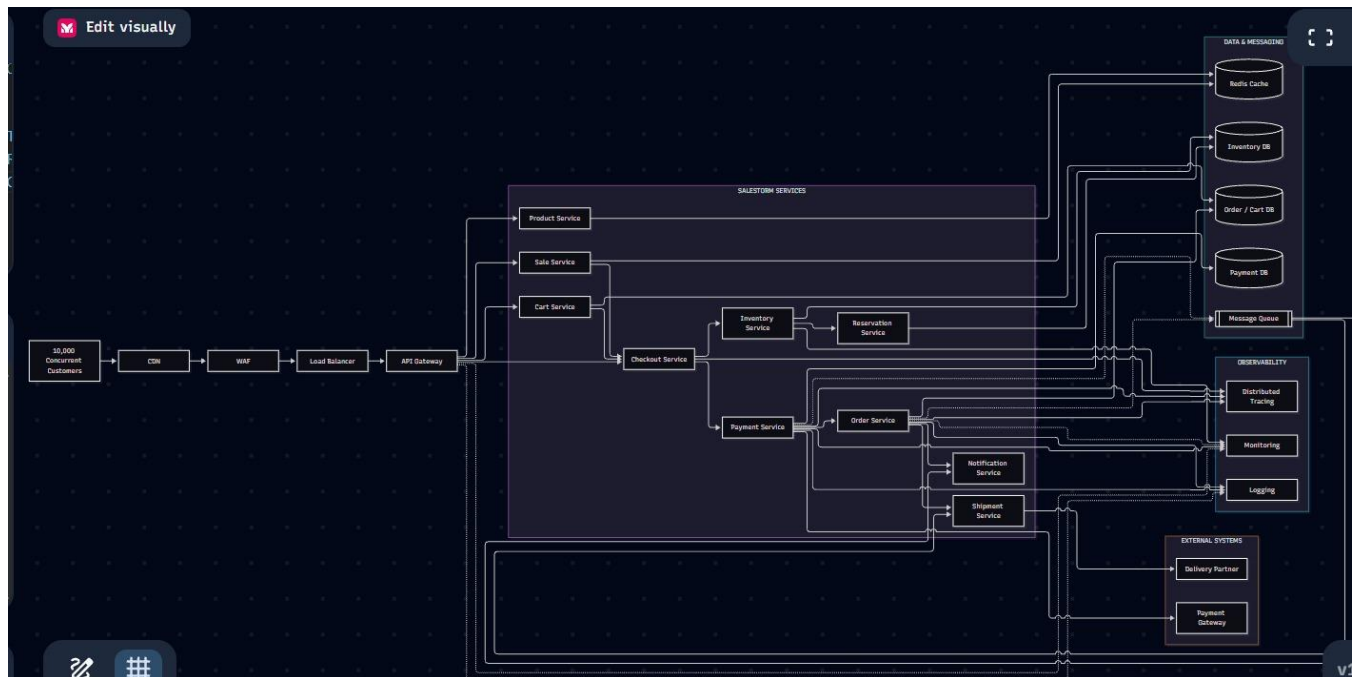
Recommended request path:

Customer → CDN/WAF → Load Balancer → API Gateway → Product / Cart / Sale Services → Inventory & Reservation → Checkout → Payment → Order → Shipment / Notification

The architecture follows a hybrid communication model. Customer-facing validation and reservation steps can be synchronous where an immediate result is required. Durable downstream workflows such as order recovery, notifications and shipment events can be asynchronous.

Component	Primary Responsibility
CDN/WAF	Edge delivery, basic protection and abuse filtering.
Load Balancer	Distribute incoming traffic across application instances.
API Gateway	Routing, authentication, rate limiting and API policy enforcement.
Product Service	Product catalogue and product availability information.
Cart Service	Customer cart and cart items.
Sale Service	Sale/deal/coupon rules.
Inventory/Reservation Service	Concurrency-safe reservation and inventory state transitions.
Checkout Service	Coordinates reservation, payment and order workflow.
Payment Service	Idempotent payment processing and gateway

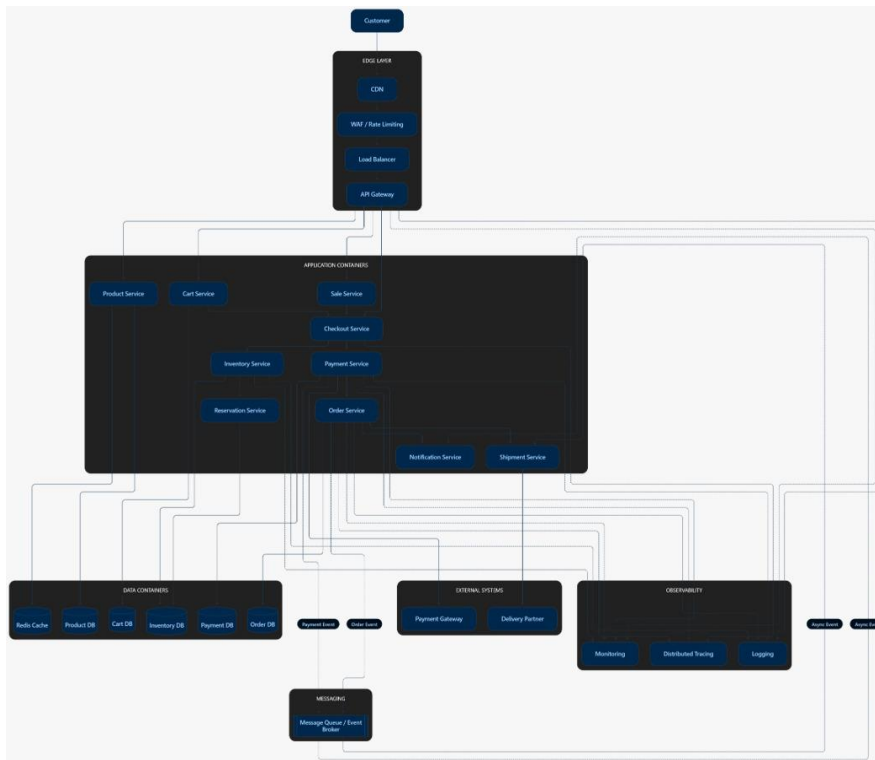
	integration.
Order Service	Order creation and lifecycle management.
Message Broker	Durable asynchronous events and retries.
Cache	Reduce repeated reads for suitable non-critical data.
Database	Transactional source of truth for inventory, reservations, payments and orders.
Monitoring/Logging/Tracing	Operational visibility and transaction tracing.



11. Container / Service Architecture

The prototype may implement these as logical modules rather than independently deployed microservices. The production-oriented design keeps the service boundaries explicit while avoiding unnecessary prototype complexity.

Logical containers: Product Service, Cart Service, Sale Service, Inventory Service, Reservation Service, Checkout Service, Payment Service, Order Service, Shipment Service, Notification Service, Message Broker, Cache and Database.



12. Critical Inventory Design

12.1 The Overselling Problem

Suppose `available_quantity = 1` and two requests arrive simultaneously. If both requests read the value before either update commits, both can believe that stock is available. A safe design must serialize access to the critical inventory record or otherwise use an equivalent concurrency-control mechanism.

12.2 Selected Prototype Strategy

The prototype uses a transactional relational database with row-level locking for the inventory record. The reservation transaction locks the relevant inventory row, checks availability, updates quantities and creates the reservation before committing.

Conceptual transaction:

BEGIN → lock inventory row → check `available_quantity` → update available/reserved quantities → create reservation with unique idempotency key → COMMIT.

12.3 Invariants

- `available_quantity` must never become negative.
- Successful reservations must never exceed available stock.
- Duplicate idempotency keys must not create duplicate reservations.
- Inventory transitions must occur inside defined transaction boundaries.
- Inventory and reservation state must remain consistent after failures.

12.4 Reservation Lifecycle

AVAILABLE → RESERVED → PAYMENT_PENDING → CONFIRMED → SOLD

Failure paths: RESERVED → PAYMENT_FAILED → RELEASED; RESERVED → TIMEOUT → RELEASED.

12.5 Reservation Expiry

Each reservation has `expires_at`. A background worker or scheduled process identifies expired reservations, transitions them to `RELEASED` and returns the reserved quantity to available inventory using a concurrency-safe transaction.

13. Idempotency Design

Idempotency prevents repeated client requests, network retries and duplicate message deliveries from creating duplicate business transactions.

Operation	Idempotency Mechanism
Purchase/Reservation	Unique <code>idempotency_key</code> on reservation.
Payment	Unique transaction reference / payment idempotency key.
Order creation	Unique reservation/payment reference and idempotent event processing.
Message consumer	Record processed event/message identifier before completing duplicate work.

When a repeated idempotency key arrives, the service returns the existing result instead of creating a new business transaction.

14. Payment Design

The payment component is modelled as an external dependency behind a `Payment` interface. This allows the prototype to simulate success, failure, timeout and gateway unavailability without using real payment infrastructure.

Scenario	Required Response
Payment succeeds	Confirm payment and proceed toward order creation.
Payment fails	Release reservation and update the business state.
Payment timeout	Perform safe retry/reconciliation without creating a duplicate transaction.
Duplicate payment	Return the existing payment result using idempotency.
Payment succeeds; Order Service fails	Retain successful payment, publish/store recovery work, retry order creation and reconcile.

15. Order Lifecycle

Recommended order state flow:

`CREATED` → `PAYMENT_PENDING` → `CONFIRMED` → `PROCESSING` → `SHIPPED` → `OUT_FOR_DELIVERY` → `DELIVERED`

Cancellation/failure transitions must be explicitly defined. Order creation after successful payment should be idempotent and recoverable.

16. Failure and Recovery Design

Failure	Response
Payment failure	Release reservation; record payment failure.
Payment timeout	Retry safely using the same idempotency key; reconcile final gateway status.

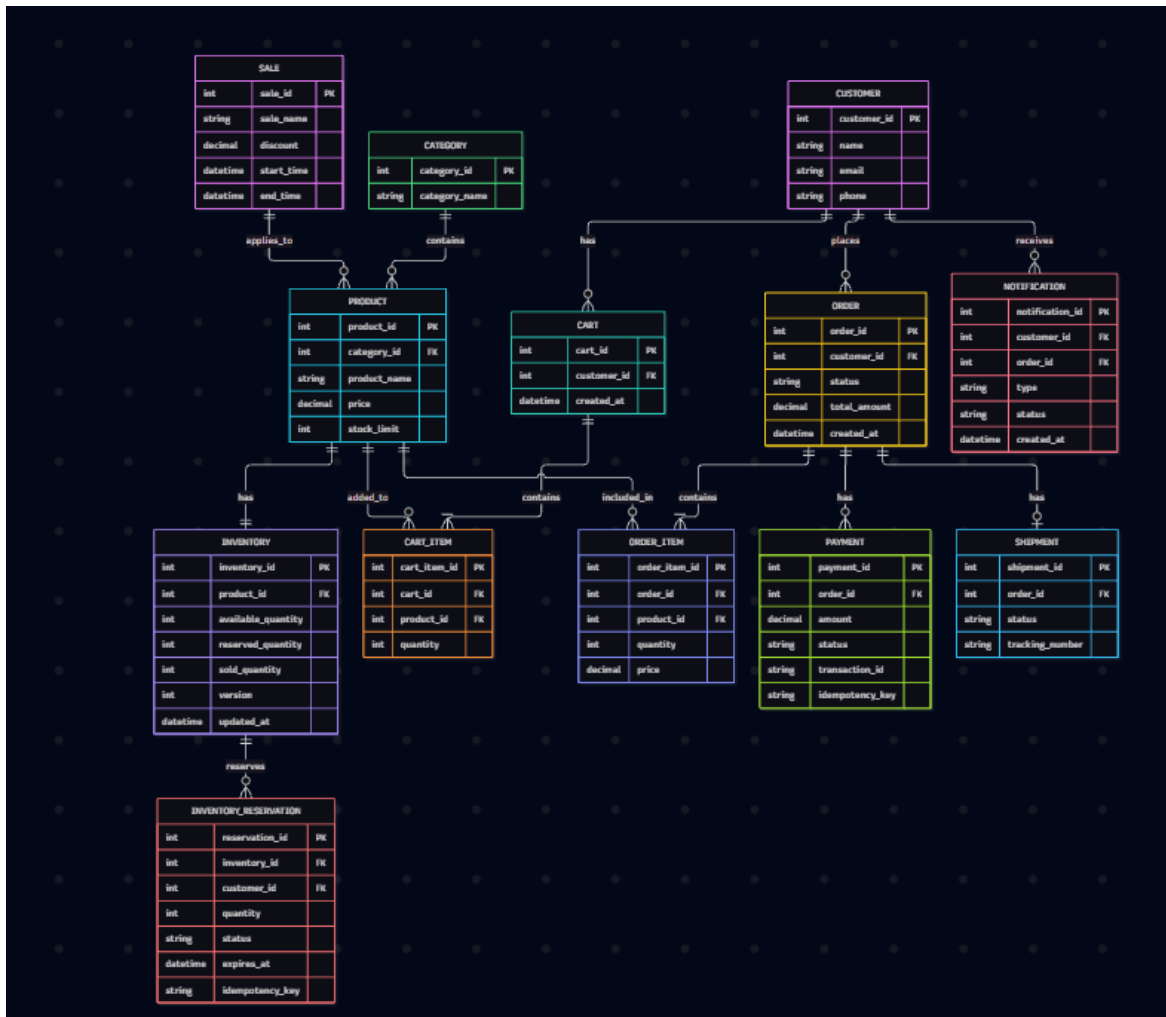
Duplicate payment	Return existing transaction; do not charge again.
Order Service unavailable	Persist/eventual retry; retain successful payment; create order after recovery.
Message processing failure	Retry; after retry limit, move to dead-letter handling and alert.
Reservation timeout	Release reserved inventory transactionally.
Database failure	Retry only where safe; rely on transaction rollback and reconciliation for distributed effects.
External gateway failure	Timeout, retry policy and circuit breaker to prevent cascading failure.

17. Database / ER Design

Core entities are CUSTOMER, PRODUCT, CATEGORY, INVENTORY, INVENTORY_RESERVATION, CART, CART_ITEM, ORDER, ORDER_ITEM, PAYMENT, SALE/DEAL/COUPON, SHIPMENT and NOTIFICATION.

Entity	Important Fields
CUSTOMER	customer_id, name, account information
PRODUCT	product_id, category_id, name, price
CATEGORY	category_id, name
INVENTORY	inventory_id, product_id, available_quantity, reserved_quantity, sold_quantity, version, updated_at
INVENTORY_RESERVATION	reservation_id, product_id, customer_id, quantity, status, expires_at, idempotency_key, timestamps
CART	cart_id, customer_id, status
CART_ITEM	cart_item_id, cart_id, product_id, quantity
ORDER	order_id, customer_id, reservation_id, payment_id, status, timestamps
ORDER_ITEM	order_item_id, order_id, product_id, quantity, price
PAYMENT	payment_id, reservation_id, customer_id, amount, status, transaction_reference, idempotency_key, timestamps
SALE/DEAL/COUPON	promotion identifiers and rules
SHIPMENT	shipment_id, order_id, status, tracking information
NOTIFICATION	notification_id, order_id/customer_id, type, status

Database requirements: primary keys, foreign keys, indexes, UNIQUE constraints, transaction boundaries, concurrency controls, consistency rules and audit information.



18. Key Database Constraints

- UNIQUE(reservation.idempotency_key).
- UNIQUE(payment.idempotency_key) and/or UNIQUE(payment.transaction_reference).
- Foreign keys from reservations to products/customers and orders to reservations/payments.
- Indexes on product_id, reservation status, expires_at, payment transaction reference and order status where useful.
- Inventory quantity constraints should prevent negative quantities at the database/business layer.
- Critical inventory updates must execute inside a transaction.

19. API Specification

Method	Endpoint	Purpose
GET	/health	Health check.
GET	/api/inventory/{product_id}	Read current inventory state.
POST	/api/products/{product_id}/reserve	Create or retrieve an idempotent reservation.
POST	/api/reservations/{reservation_id}/pay	Process payment for a reservation.
POST	/api/reservations/{reservation_id}/expire	Prototype/admin action to simulate expiry.
POST	/api/orders/create	Create/recover an order

		idempotently.
GET	/api/orders/{order_id}	Read order status.
GET	/api/reservations/{reservation_id}	Read reservation status.
POST	/api/simulation/flash-sale	Run the 10,000-request simulation.

19.1 Example Reservation Request

POST /api/products/1/reserve

Headers: Idempotency-Key: BUY-CUSTOMER-123-PRODUCT-1

Body: {"customer_id":123,"quantity":1}

19.2 Example Success Response

{"reservation_id":"R123","status":"RESERVED","quantity":1,"expires_at":"..."}

19.3 Example Out-of-Stock Response

{"error":"OUT_OF_STOCK","message":"No inventory available"}

20. Low-Level Design

Critical modules: Inventory/Reservation, Payment and Order. Each module should expose focused interfaces and keep persistence, domain rules and external-provider integration separated.

Module	Main Responsibilities
InventoryService	Validate stock, acquire concurrency control, update quantities.
ReservationService	Create, confirm, expire and release reservations.
PaymentService	Idempotent payment, timeout/retry and provider abstraction.
OrderService	Create and transition orders, handle idempotent recovery.
CheckoutService	Orchestrate the overall purchase flow.
Repository layer	Persistence abstraction for entities.
Event/Message layer	Asynchronous recovery and notification events.

21. Sequence – Successful Purchase

Customer → API Gateway → Checkout → Reservation → Inventory DB → Checkout → Payment → Payment Gateway → Payment → Order → Order DB → Customer.

The reservation is established before payment. Payment success transitions the reservation toward confirmation/sale and triggers idempotent order creation.

22. Sequence – Payment Failure

Customer → Checkout → Reservation → Payment → Payment Gateway → FAILURE → Payment Service → Release Reservation → Inventory DB → Failure response to Customer.

23. Sequence – Payment Success + Order Failure

Customer → Payment → Payment Gateway → SUCCESS → Payment DB → Order Service → UNAVAILABLE
→ Event/Retry Queue → Order Service RECOVERS → Create Order → Mark workflow complete.

24. State Model

State	Meaning
AVAILABLE	Inventory unit is available for reservation.
RESERVED	Inventory is temporarily held for a customer.
PAYMENT_PENDING	Payment is being processed.
CONFIRMED	Payment has succeeded and purchase is confirmed.
SOLD	Inventory is finalized as sold.
PAYMENT_FAILED	Payment did not succeed.
TIMEOUT	Reservation/payment exceeded the permitted time.
RELEASED	Reserved quantity has been returned to availability.

25. SOLID Mapping

Principle	Application
SRP	Payment processing, order management, reservation and notification have separate responsibilities.
OCP	New payment or pricing strategies can be introduced through abstractions.
LSP	Provider implementations conform to their interfaces without breaking callers.
ISP	Use focused interfaces for payment, inventory, repository and notification capabilities.
DIP	High-level checkout logic depends on service/provider abstractions rather than concrete external providers.

26. Design Patterns

Pattern	Application	Benefit
Strategy	Payment, pricing or delivery strategy.	Add/replace strategies without changing core orchestration.
Factory	Create payment/provider objects.	Centralize provider selection/creation.
State	Reservation/order lifecycle.	Keep state transitions explicit.
Observer	Order events and notifications.	Decouple notification consumers.
Adapter	External payment/delivery providers.	Hide provider-specific APIs.
Repository	Persistence access.	Separate domain logic from database implementation.
Facade	Checkout orchestration.	Provide a simple entry point to a multi-step workflow.
Circuit Breaker	Payment/external services.	Prevent repeated calls to an unhealthy dependency.

27. Scalability Design

The architecture should support horizontal scaling at stateless API/service layers. The API Gateway and Load Balancer distribute traffic. Cache is used for suitable read-heavy, non-critical data. Queueing absorbs asynchronous work and prevents downstream services from being overwhelmed. Inventory remains a consistency-critical boundary and requires controlled contention.

The brief identifies approximately 10,000 requests/sec as normal traffic and asks the architecture to reason about up to 500,000 requests/sec for flash-sale conditions. These figures are architectural targets, not claims that the prototype itself achieves those production rates.

Potential Bottleneck	Mitigation
API instances	Horizontal scaling and load balancing.
Inventory row contention	Short transactions, controlled admission/queueing and careful partitioning strategy.
Database reads	Caching and read scaling where consistency permits.
Message backlog	Consumer scaling, monitoring and backpressure.
Payment gateway	Timeouts, circuit breaker and asynchronous recovery.
Order Service	Horizontal scaling and event-based retry.

28. Security Design

- Authentication and authorization at API boundaries.
- HTTPS for service/client communication.
- Rate limiting and abuse protection during flash-sale traffic.
- Strict request validation.
- Do not store raw payment credentials in the prototype.
- Secrets stored outside source code.
- Audit logs for critical inventory, payment and order transitions.
- Least-privilege access to database and infrastructure.

29. Observability Design

Metric / Signal	Purpose
Request rate	Measure incoming traffic.
Request latency	Detect performance degradation.
Error rate	Detect service failures.
Reservation failures	Identify inventory contention/out-of-stock behaviour.
Payment failures	Track payment reliability.
Order conversion	Measure successful purchase completion.
Queue backlog	Detect asynchronous processing delay.
Inventory inconsistency alert	Detect critical correctness failure.
Distributed trace	Follow checkout → payment → order across services.

Structured logs should include correlation/request ID, customer ID where appropriate, reservation ID, payment ID, order ID, event type, timestamp, status and error information.

30. Architecture Trade-offs / ADR

Decision	Alternative	Selected	Reason / Trade-off
SQL vs NoSQL	NoSQL	Relational SQL	Strong transactional semantics and constraints are valuable for inventory and payment records.
Optimistic vs pessimistic concurrency	Optimistic	Row-level locking for prototype	The inventory row is a high-contention correctness boundary; locking makes the critical section explicit. Trade-off: contention can reduce throughput.
Sync vs async	Fully synchronous	Hybrid	Immediate reservation/payment decisions remain simple while recovery and downstream work benefit from asynchronous processing.
Direct retry vs idempotent retry	Direct retry	Idempotent retry	Prevents duplicate payment/order effects. Requires persistent idempotency records.
Cache inventory vs source-of-truth DB	Cache	DB for critical reservation	Stale inventory cannot decide final stock allocation. Trade-off: more database contention.

31. Prototype Specification

The prototype is a lightweight validation system. It should not attempt to implement the entire production architecture. It should implement enough of Inventory/Reservation, Payment, Order and Simulation to validate the most important correctness assumptions.

31.1 Suggested Technology

Layer	Technology
Backend	Python 3.10+ / FastAPI
Database	PostgreSQL
ORM	SQLAlchemy
Testing	Pytest
Simulation	Python concurrent requests / async or controlled worker pool
Frontend	React (simple dashboard)
Documentation	Markdown + PDF/Word
Diagrams	Draw.io / Mermaid / PlantUML

31.2 Suggested Project Structure

```
backend/  
  app/  
    main.py  
    models/  
    schemas/  
    services/  
      inventory_service.py  
      reservation_service.py  
      payment_service.py  
      order_service.py  
      checkout_service.py  
    repositories/  
    workers/  
    simulation/  
    tests/  
frontend/  
docs/  
  architecture.md  
  concurrency.md  
  failure-handling.md  
  api.md
```

32. Prototype Scenarios

Scenario	Input	Expected Validation
S1 Basic purchase	1 customer, 1 unit	Reservation/payment/order succeed.
S2 100 concurrent	100 users, 100 units	At most 100 successful reservations.
S3 Flash sale	10,000 users, 100 units	At most 100 successful reservations; no overselling.
S4 Duplicate purchase	Same idempotency key twice	One reservation only.
S5 Payment failure	Reservation + failed payment	Reservation released.
S6 Reservation expiry	Reservation passes expiry	Reserved quantity released.
S7 Duplicate payment	Same payment key twice	One payment transaction.
S8 Order Service down	Payment succeeds, order unavailable	Recovery event/retry eventually creates order.
S9 Inventory zero	Purchase after stock exhausted	OUT OF STOCK.
S10 Payment timeout	Gateway timeout + retry	No duplicate payment transaction.

33. Flash-Sale Simulation

Run: python run_simulation.py

Expected console structure:

SALESTORM FLASH SALE SIMULATION

Stock: 100

Concurrent Requests: 10,000

FINAL RESULT

Total Requests: [ACTUAL]
Successful Reservations: [ACTUAL]
Out of Stock: [ACTUAL]
Duplicates: [ACTUAL]
Payment Success: [ACTUAL]
Payment Failed: [ACTUAL]
Orders Created: [ACTUAL]
Final Available: [ACTUAL]
Final Reserved: [ACTUAL]
Final Sold: [ACTUAL]
Overselling Detected: [TRUE/FALSE]

Do not pre-fill the actual values. Insert the values from the validated prototype run.

34. Prototype Validation Evidence

Insert screenshots after implementation:

1. Flash-sale dashboard showing 10,000 requests and final inventory result.
2. Database inventory row before and after the simulation.
3. Duplicate idempotency request and returned existing result.
4. Payment failure followed by reservation release.
5. Reservation expiry followed by inventory restoration.
6. Payment success while Order Service is unavailable.
7. Retry/event processing followed by successful order creation.
8. Automated test report.

35. Results and Analysis

Test	Expected	Actual	Status	Evidence
10,000 concurrent requests	≤ 100 successful reservations	[FILL]	[FILL]	Screenshot/Test ID
Overselling	0	[FILL]	[FILL]	Simulation output
Duplicate reservation	0 duplicate reservations	[FILL]	[FILL]	API/test
Payment failure release	Inventory restored	[FILL]	[FILL]	Screenshot
Reservation expiry	Inventory restored	[FILL]	[FILL]	Screenshot
Duplicate payment	1 transaction	[FILL]	[FILL]	Test
Order Service failure	Eventual order creation	[FILL]	[FILL]	Logs/screenshot
Zero inventory	OUT_OF_STOCK	[FILL]	[FILL]	API/test

Analysis template: The validation demonstrates whether the proposed concurrency-control and idempotency mechanisms preserve the inventory invariants under concurrent load. Any failure must be documented, investigated and corrected before the final demonstration.

36. Failure Injection Demonstration

The final demo should walk through five critical cases: (1) successful purchase, (2) failed payment and reservation release, (3) duplicate Buy request and idempotent behaviour, (4) payment success followed by temporary Order Service failure, and (5) inventory reaching zero.

For each case, record request ID/correlation ID, relevant reservation/payment/order IDs, state transitions and final outcome.

37. Limitations

- The prototype is not a production deployment and does not prove production throughput.
- Payment and external services are simulated.
- Actual distributed deployment, network partitions and infrastructure-level failures may not be fully reproduced.
- Production database scaling and inventory sharding strategies require additional benchmarking.
- Load-test results depend on the local hardware and environment.

38. Future Enhancements

- Deploy services independently with container orchestration.
- Introduce production-grade distributed rate limiting and admission control.
- Benchmark alternative inventory allocation strategies.
- Introduce database partitioning/sharding where justified.
- Add production-grade message broker and dead-letter infrastructure.
- Add distributed tracing backend and dashboards.
- Integrate a real payment provider in a controlled test environment.
- Perform sustained load testing and capacity planning.

39. AI-Assisted Development Note

AI tools were used as development accelerators for the optional prototype, test generation, simulation scripts, boilerplate and documentation support. The team remains responsible for requirement analysis, architecture selection, concurrency reasoning, inventory consistency decisions, payment/order state modelling, security and reliability decisions, validation and final technical explanation.

AI-generated code was reviewed and tested before inclusion in the prototype. The prototype is evidence supporting the architecture and is not treated as a substitute for system design.

AI tool(s) used: [INSERT TOOL NAME(S)]

Primary prompts used: [INSERT FINAL PROMPT / PROMPT SUMMARY]

Validation performed: [INSERT TESTS AND SIMULATION RESULTS]

40. Final Jury Walkthrough

9. Start with the business problem: 10,000 customers competing for 100 units.
10. Show the complete HLD and identify the critical inventory consistency boundary.
11. Explain how requests are admitted, routed and controlled under high concurrency.
12. Trace one successful purchase from Buy Now through reservation, payment and order.
13. Trace payment failure and show reservation release.
14. Trace duplicate request and show idempotency.
15. Trace payment success followed by Order Service failure and recovery.
16. Show the final simulation evidence proving that successful sales did not exceed available inventory.
17. Explain the major trade-offs: consistency vs availability, synchronous vs asynchronous processing and database simplicity vs horizontal scalability.

41. Final Deliverable Checklist

Deliverable	Status
Requirements & Assumptions Document	☐
System Context Diagram	☐
HLD Architecture	☐
Container Diagram	☐
Component Diagram	☐
Deployment Diagram	☐
Database / ER Diagram	☐
Class Diagram	☐
Purchase / Reservation Sequence Diagram	☐
Payment Sequence Diagram	☐
Order Sequence Diagram	☐
Order / Reservation State Diagram	☐
SOLID Mapping	☐
Design Pattern Mapping	☐
API Specification	☐
Scalability & Reliability Design	☐
Security & Observability Design	☐
Architecture Decision Record	☐
AI-Assisted Prototype / Simulation Evidence	☐
AI Usage Note / Prompt Summary	☐
Final Presentation	☐

42. Implementation Guidance – Build in This Order

18. Finalize requirements and assumptions.
19. Create the HLD and service boundaries.
20. Create the inventory/reservation state model.
21. Create the ER diagram and database constraints.
22. Implement Inventory/Reservation with transactional concurrency control.
23. Implement idempotency.
24. Implement simulated Payment Service.
25. Implement Order Service and recovery path.
26. Implement the flash-sale simulation.
27. Write automated concurrency and failure tests.
28. Run the 10,000-request simulation.
29. Investigate and fix every correctness failure.
30. Capture screenshots/logs/results.
31. Update the Results section with actual evidence.
32. Create the final 5-minute presentation.

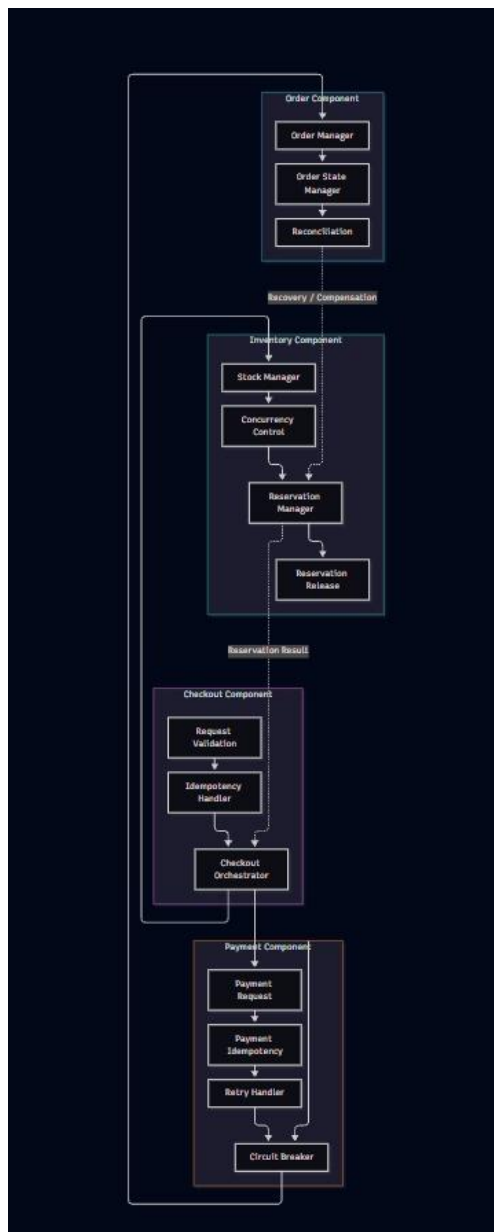
43. Reference to Hackathon Evaluation

The evaluation gives the greatest weight to high-level architecture, concurrency/scalability, inventory reservation/consistency and payment/order management. The prototype/validation itself has a smaller weighting, so the team should prioritize a defensible architecture and clear technical reasoning over a large codebase.

Evaluation Area	Weight
Business & Requirement Understanding	10%

High-Level Architecture & Service Boundaries	15%
High Concurrency & Scalability Design	15%
Inventory Reservation & Consistency	15%
Payment & Order Management Design	10%
LLD & UML	10%
SOLID & Design Patterns	5%
Database, API & Event Design	5%
Reliability, Security & Observability	5%
Architecture Trade-offs & Defense	5%
AI-Assisted Prototype / Validation	3%
Final Pitch & Team Understanding	2%

44. Component Diagram



45. Class Diagram



44. Conclusion

SALESTORM is designed as a scalable and defensible flash-sale architecture in which correctness of limited inventory is treated as a critical consistency boundary. The architecture combines controlled concurrency, temporary reservation, idempotency, reliable payment processing, asynchronous recovery, explicit state transitions and operational observability.

The prototype should provide measurable evidence for the most important assumptions without attempting to replace the system design. The final submission should therefore present the architecture, design decisions, failure handling and trade-offs first, followed by the prototype and its validated results.